

# CATCHFOOD

## Guía completa de código para defensa oral

**De cero** a una explicación técnica segura

**Objetivo:** que puedas leer cada construcción importante del proyecto y explicar qué es, qué hace y por qué se utilizó.

Incluye Kotlin, Jetpack Compose, ViewModel, Canvas, corrutinas, navegación, Room/SQLite, audio, colisiones y rendimiento.

*Preparado a partir del código real de feature/catchfood.*

## Cómo estudiar esta guía hoy

**Regla principal:** no memorices líneas completas. Memorizá responsabilidades y recorridos de datos.

1. Leé primero el resumen de 90 segundos y repetilo en voz alta.
2. Aprendé el flujo: UI → ViewModel → GameLogic → UiState → UI → Room.
3. Estudiá el glosario de Kotlin y Compose. Ahí están lambda, callback, state, suspend y demás términos.
4. Revisá los fragmentos línea por línea y explicalos sin mirar.
5. Terminá con las preguntas de jurado y el simulacro oral.

## Resumen oral de 90 segundos

**Respuesta lista para decir:** CatchFood es un minijuego Android hecho con Jetpack Compose. La interfaz está separada del estado mediante CatchFoodViewModel y CatchFoodUiState. El ViewModel coordina el motor CatchFoodGameLogic, que actualiza posiciones, velocidad, colisiones y reglas de derrota. El loop se sincroniza con los frames reales usando withFrameNanos y delta time. Para evitar lag, los objetos se dibujan juntos en un Canvas, en lugar de usar muchos componentes Image que se reubiquen cada frame. Al terminar una partida, el ViewModel guarda el puntaje mediante un Repository en Room, que utiliza SQLite local. El histórico ejecuta una consulta SQL ordenada y muestra los diez mejores resultados.

## Mapa mental

Entrada táctil → lambda de arrastre → ViewModel.movePlayer() → GameLogic.playerX → Canvas redibuja el conejo.

Frame nuevo → ViewModel.updateGame() → GameLogic.updateGame() → colisión/puntaje → UiState cambia → Compose actualiza contadores.

Derrota → ViewModel captura finalScore → corrutina → Repository → DAO → Room → SQLite.

Histórico → produceState → Repository → DAO → SELECT ORDER BY LIMIT 10 → LazyColumn.

# 1. Fundamentos de Kotlin que aparecen en CatchFood

## Palabras reservadas y operadores

| Término           | Qué significa y cómo defenderlo                                                                   |
|-------------------|---------------------------------------------------------------------------------------------------|
| <b>val</b>        | Declara una referencia que no puede reasignarse. El objeto interno sí podría ser mutable.         |
| <b>var</b>        | Declara una variable cuyo valor puede cambiar.                                                    |
| <b>fun</b>        | Declara una función: una unidad de código que puede recibir parámetros y devolver un resultado.   |
| <b>class</b>      | Define un tipo con datos y comportamiento.                                                        |
| <b>data class</b> | Clase orientada a datos. Kotlin genera equals, hashCode, toString, componentN y copy.             |
| <b>object</b>     | Crea una única instancia. En navegación funciona como destino singleton.                          |
| <b>private</b>    | El elemento solo puede usarse dentro del alcance permitido, por ejemplo la misma clase o archivo. |
| <b>override</b>   | Indica que una propiedad o función reemplaza una declaración heredada.                            |
| <b>null</b>       | Representa ausencia de valor.                                                                     |
| <b>?</b>          | Convierte un tipo en nullable: puede contener un valor o null.                                    |
| <b>?:</b>         | Operador Elvis: usa el valor derecho si el izquierdo es null.                                     |
| <b>?.</b>         | Llamada segura: ejecuta el acceso solo si el valor no es null.                                    |
| <b>::</b>         | Referencia a función o propiedad; no la ejecuta todavía.                                          |
| <b>by</b>         | Delegación. Permite leer/escribir un State como si fuera una variable normal.                     |

## Lambda

Una lambda es una función sin nombre que puede guardarse, pasarse como parámetro o ejecutarse después.

Ejemplo real del menú

```
onClick = { onGoToGame() }
```

- `onClick` espera una función que no recibe parámetros y no devuelve información útil: `() -> Unit`.
- Las llaves `{ ... }` contienen el cuerpo de la lambda.
- La lambda no se ejecuta al construir la pantalla; se ejecuta cuando el usuario toca el botón.

**Respuesta oral:** Una lambda es una función anónima que puedo pasar como dato. En CatchFood la uso para comunicar eventos como clics, volver o mover al jugador.

## Callback

Un callback es una función que se entrega a otro componente para que la llame cuando ocurra un evento.

```
fun CatchFoodScreen(
    onBack: () -> Unit,
    onGoToGame: () -> Unit,
    onGoToLeaderboard: () -> Unit
)
```

- La pantalla no conoce NavController.
- Solo sabe que puede llamar `onGoToGame()`.
- El NavHost decide qué navegación concreta ocurre.

**Ventaja:** reduce el acoplamiento y hace que la pantalla sea más reutilizable y fácil de probar.

## Unit

`Unit` es el equivalente conceptual a `void`: indica que la función termina una acción pero no devuelve un resultado relevante.

## Genéricos

Los símbolos `` permiten que una clase o función trabaje con distintos tipos. Ejemplo: `List<Int>` es una lista cuyo elemento debe ser Int; `List<FallingObject>` contiene objetos que caen.

## Funciones de orden superior

Son funciones que reciben o devuelven funciones. ``forEach``, ``use``, ``remember``, ``LaunchedEffect`` y ``detectDragGestures`` reciben lambdas.

## 2. Jetpack Compose desde cero

| Término              | Qué significa y cómo defenderlo                                                               |
|----------------------|-----------------------------------------------------------------------------------------------|
| <b>@Composable</b>   | Anotación que marca una función capaz de describir interfaz declarativa.                      |
| <b>Composición</b>   | Proceso inicial en el que Compose ejecuta funciones y construye el árbol de UI.               |
| <b>Recomposición</b> | Reejecución selectiva cuando cambia un estado observado.                                      |
| <b>State</b>         | Valor observable. Cuando cambia, Compose invalida la fase donde fue leído.                    |
| <b>Modifier</b>      | Cadena de instrucciones para tamaño, posición, entrada táctil, fondo y otros comportamientos. |
| <b>dp</b>            | Unidad independiente de densidad usada para tamaños y posiciones visuales.                    |
| <b>sp</b>            | Unidad para texto que considera densidad y preferencias de accesibilidad.                     |
| <b>Context</b>       | Acceso a recursos y servicios Android; aquí se usa para assets, MediaPlayer y Room.           |

### Qué significa UI declarativa

En una UI imperativa ordenarías manualmente: busca el Text y cambia su contenido. En Compose declararás cómo debe verse la UI para un estado dado:

```
Text("Comida: ${uiState.score}")
```

Cuando `uiState.score` cambia, Compose vuelve a ejecutar la parte necesaria y genera el texto nuevo.

### remember

```
val bitmaps = remember(context, density) {
    CatchFoodBitmaps.load(context, density)
}
```

- `remember` conserva el resultado entre recomposiciones.
- `context` y `density` son claves. Si cambia una, se vuelve a calcular.
- Sin remember, los PNG podrían decodificarse repetidamente.

**Respuesta oral:** remember evita repetir trabajo durante recomposiciones, pero no reemplaza al ViewModel ni garantiza persistencia después de destruir la pantalla.

## Estado y delegación by

```
var uiState by mutableStateOf(gameLogic.toUiState())
private set
```

- `mutableStateOf` crea un contenedor observable.
- `by` delega get y set: permite escribir `uiState.score` en vez de `uiState.value.score`.
- `private set` permite lectura pública, pero solo el ViewModel puede cambiarlo.

## Efectos

| Término                 | Qué significa y cómo defenderlo                                                                                |
|-------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>LaunchedEffect</b>   | Inicia una corrutina ligada a la vida del Composable. Se cancela cuando sale de composición o cambia su clave. |
| <b>DisposableEffect</b> | Ejecuta inicialización y exige limpieza mediante onDispose.                                                    |
| <b>produceState</b>     | Ejecuta trabajo suspendido y expone el resultado como State de Compose.                                        |

### 3. Arquitectura real del módulo

El módulo no implementa MVVM de forma académicamente pura, pero aplica una separación clara inspirada en MVVM y Repository:

- View: CatchFoodScreen, CatchFoodGame y CatchFoodLeaderboard.
- ViewModel: CatchFoodViewModel.
- Model/dominio: CatchFoodGameLogic, FallingObject y CatchFoodUiState.
- Datos: Entity, DAO, Database y Repository.
- Servicios: CatchFoodAudioManager.
- Coordinación: CatchFoodNavHost y destinos.

**Frase de defensa:** La UI no guarda directamente en SQLite ni implementa las reglas. Envía eventos al ViewModel, observa UiState y delega persistencia al Repository.

#### Flujo de dependencias

6. CatchFoodGame obtiene el ViewModel con ``viewModel()``.
7. El ViewModel posee una instancia de CatchFoodGameLogic.
8. GameLogic actualiza el mundo del juego.
9. El ViewModel sincroniza valores visibles hacia CatchFoodUiState.
10. Compose lee UiState y dibuja contadores/menús.
11. Al perder, el ViewModel usa Repository para guardar en Room.

#### Por qué separar responsabilidades

- Cambiar el diseño no obliga a reescribir SQL.
- Cambiar la base no obliga a reescribir el Canvas.
- Las reglas pueden estudiarse o probarse de forma aislada.
- El estado no desaparece por una simple recomposición.



## 4. Menú principal línea por línea

```
@Composable
fun CatchFoodScreen(
    onBack: () -> Unit,
    onGoToGame: () -> Unit,
    onGoToLeaderboard: () -> Unit
) {
```

| Término                | Qué significa y cómo defenderlo                                        |
|------------------------|------------------------------------------------------------------------|
| <b>@Composable</b>     | Le dice al compilador de Compose que la función puede emitir interfaz. |
| <b>fun</b>             | Inicia la declaración de una función.                                  |
| <b>CatchFoodScreen</b> | Nombre de la función/pantalla.                                         |
| <b>onBack</b>          | Parámetro que almacena una función para volver.                        |
| <b>:</b>               | Separa el nombre del parámetro de su tipo.                             |
| <b>() -&gt; Unit</b>   | Tipo función: cero parámetros, devuelve Unit.                          |

### Contexto y audio

```
val context = LocalContext.current
val audioManager = remember { CatchFoodAudioManager(context) }

DisposableEffect(Unit) {
    audioManager.startMusic()
    onDispose { audioManager.stopMusic() }
}
```

- `LocalContext.current` obtiene el Context disponible en el árbol Compose.
- `remember` mantiene el mismo AudioManager durante recomposiciones.
- `DisposableEffect(Unit)` tiene una clave constante: se inicia una vez para esa entrada en composición.
- `onDispose` se ejecuta al abandonar el menú.

### Carga segura del fondo

```
val fondoCompleto = remember {
    try {
        context.assets.open(
            "games/catchfood/sprites/background_historico.png"
        ).use { inputStream ->
```

```

        BitmapFactory.decodeStream(inputStream).asImageBitmap()
    }
} catch (e: Exception) {
    null
}
}

```

- `try/catch` evita cerrar la aplicación si falla la lectura.
- `assets.open` abre un `InputStream`.
- `use` garantiza que el stream se cierre incluso si ocurre una excepción.
- `decodeStream` crea un `Bitmap` de Android.
- `asImageBitmap` lo adapta para Compose.
- El tipo inferido es `ImageBitmap?` porque el catch devuelve null.

## let y null safety

```

fondoCompleto?.let { bitmap ->
    Image(bitmap = bitmap, ...)
}

```

`?.let` ejecuta la lambda solo si `fondoCompleto` no es null. Dentro de la lambda, `bitmap` ya no es nullable.

## 5. Navegación y lambdas

```
@Serializable  
object CatchFoodGameScreen
```

- `@Serializable` permite convertir el destino a una representación navegable.
- `object` crea una única instancia porque la ruta no necesita parámetros.

```
composable<CatchFoodGameScreen> {  
    CatchFoodGame(  
        onBack = { navController.popBackStack() }  
    )  
}
```

- `composable<T>` registra una pantalla para el tipo T.
- La lambda exterior es el contenido que Navigation mostrará.
- La lambda `onBack` se pasa como callback.
- `popBackStack()` elimina la pantalla actual y revela la anterior.

**Pregunta trampa:** ¿Por qué no pasar NavController a todas las pantallas? Porque aumentaría el acoplamiento. Las pantallas solo necesitan expresar eventos; el NavHost decide las rutas.

## 6. CatchFoodUiState palabra por palabra

```
data class CatchFoodUiState(
    val score: Int = 0,
    val poisonHits: Int = 0,
    val missedFood: Int = 0,
    val playerState: String = "eat",
    val isPaused: Boolean = false,
    val isGameOver: Boolean = false
)
```

| Término            | Qué significa y cómo defenderlo                                         |
|--------------------|-------------------------------------------------------------------------|
| <b>data class</b>  | Clase de datos. Kotlin genera operaciones útiles, especialmente copy(). |
| <b>val</b>         | Cada propiedad es inmutable dentro de una instancia concreta.           |
| <b>Int</b>         | Número entero de 32 bits.                                               |
| <b>String</b>      | Cadena de caracteres; aquí identifica el sprite del conejo.             |
| <b>Boolean</b>     | Solo puede ser true o false.                                            |
| <b>= 0 / false</b> | Valor por defecto usado cuando se construye sin argumento.              |

### Qué es UiState

UiState significa User Interface State: el estado necesario para representar la pantalla en un instante.

- No es la base de datos.
- No es la pantalla.
- No contiene funciones de negocio.
- Es una fotografía inmutable de valores visibles.

```
uiState = uiState.copy(isPaused = true)
```

`copy` crea otro CatchFoodUiState copiando todo, excepto isPaused. Esta inmutabilidad hace predecibles los cambios de UI.

**Respuesta oral:** UiState es un único objeto que agrupa todo lo que la pantalla necesita observar. Cuando creo una copia con valores nuevos, Compose detecta el cambio y recompone lo necesario.

## 7. CatchFoodViewModel a nivel de código

```
class CatchFoodViewModel(application: Application)
: AndroidViewModel(application) {
```

- La clase recibe `Application`, un contexto con vida igual al proceso de la app.
- Los dos puntos indican herencia.
- Hereda de AndroidViewModel porque necesita contexto para Room.
- No guarda una Activity, evitando fugas de memoria.

```
val gameLogic = CatchFoodGameLogic()
private val scoreRepository = CatchFoodScoreRepository(application)
private var scoreSavedForCurrentGame = false
```

- El ViewModel posee el motor del juego.
- Repository es privado: la UI no accede directamente a datos.
- El booleano funciona como guardia contra inserciones duplicadas.

### Actualización y guardado

```
fun updateGame(deltaSeconds: Float) {
    gameLogic.updateGame(deltaSeconds)
    syncUiState()

    if (gameLogic.isGameOver && !scoreSavedForCurrentGame) {
        scoreSavedForCurrentGame = true
        val finalScore = gameLogic.score
        viewModelScope.launch {
            scoreRepository.saveScore(finalScore)
        }
    }
}
```

- `deltaSeconds: Float` es tiempo transcurrido con decimales.
- Primero avanza el motor; después sincroniza datos visibles.
- `&&` significa AND: ambas condiciones deben cumplirse.
- `!` niega un Boolean. `!scoreSaved...` significa que todavía no fue guardado.
- `finalScore` captura el valor antes de iniciar la corrutina; un reinicio rápido no lo cambia.
- `viewModelScope.launch` inicia una corrutina ligada al ViewModel.

### Qué es una corrutina

Es una unidad de trabajo suspendible y liviana. Permite esperar operaciones como SQLite sin bloquear la interfaz.

**No digas:** ‘una corrutina es un hilo’. Puede usar hilos, pero es una abstracción más liviana y puede suspenderse sin bloquearlos.

## Sincronización eficiente

`syncUiState()` compara cada campo antes de crear un estado nuevo. Así no genera objetos ni recomposiciones en cada frame cuando solo cambian posiciones del Canvas.

```
if (uiState.score != gameLogic.score || ...) {  
  uiState = gameLogic.toUiState(isPaused = uiState.isPaused)  
}
```

- `!=` significa diferente.
- `||` significa OR: basta que una condición sea verdadera.
- `toUiState` es una función de extensión privada sobre GameLogic.

## 8. Modelo y motor del juego

```
data class FallingObject(
    var x: Float,
    var y: Float,
    val isPoison: Boolean
)
```

- x/y son var porque cambian mientras cae o reaparece.
- isPoison es val porque el tipo del objeto no cambia durante su vida.

### Estado optimizado de Compose

```
var playerX by mutableFloatStateOf(300f)
var score by mutableIntStateOf(0)
var isGameOver by mutableStateOf(false)
```

- Las versiones especializadas Float/Int evitan boxing innecesario.
- `300f` es un literal Float; la `f` evita que Kotlin lo interprete como Double.
- Compose observa estos valores en la fase donde se leen.

### private set

```
var speed = INITIAL_SPEED
    private set
```

Cualquier código puede leer speed, pero solo CatchFoodGameLogic puede cambiarla.

### Companion object y constantes

```
private companion object {
    const val TARGET_FPS = 60f
    const val INITIAL_SPEED = 8f
}
```

- `companion object` contiene miembros asociados a la clase, no a una instancia específica.
- `const val` es una constante conocida en compilación.
- Los nombres en mayúsculas siguen la convención de constantes.

## 9. Loop, FPS y delta time

```
LaunchedEffect(gameViewModel) {
    var previousFrameNanos = 0L
    while (true) {
        withFrameNanos { frameNanos ->
            // actualización
        }
    }
}
```

- LaunchedEffect inicia el loop como corrutina y lo cancela al salir de la pantalla.
- `0L` es un literal Long; la L identifica 64 bits.
- `while (true)` crea el ciclo continuo.
- `withFrameNanos` suspende hasta que Android vaya a producir otro frame.

```
val deltaSeconds =
    (frameNanos - previousFrameNanos) / 1_000_000_000f
```

- Los timestamps vienen en nanosegundos.
- Un segundo contiene mil millones de nanosegundos.
- Los guiones bajos solo mejoran lectura del número.

```
val frameScale =
    (deltaSeconds * TARGET_FPS).coerceIn(0f, MAX_FRAME_SCALE)
```

```
obj.y += speed * frameScale
```

A 60 FPS,  $\text{delta} \approx 0.0167$  s y  $\text{frameScale} \approx 1$ . En un frame más largo, el desplazamiento aumenta proporcionalmente.

**Respuesta oral:** Delta time hace que el movimiento dependa del tiempo real y no del número de frames. `coerceIn` limita saltos si el dispositivo se congela momentáneamente.

### Pausa

Cuando `UiState` indica pausa o derrota, el loop no llama `updateGame` y reinicia `previousFrameNanos`. Al continuar no acumula todo el tiempo pausado como un salto enorme.



## 10. Spawn y cola de objetos

```
val objectTypes =
    (List(8) { false } + List(2) { true }).shuffled()
```

- `List(8) { false }` fabrica ocho valores false: zanahorias.
- `+` concatena ambas listas.
- `List(2) { true }` representa dos venenos.
- `shuffled()` devuelve una copia con orden aleatorio.

```
objectTypes.forEachIndexed { index, isPoison ->
    objects.add(
        FallingObject(
            x = Random.nextInt(80, 850).toFloat(),
            y = INITIAL_SPAWN_Y - SPAWN_SPACING_WORLD * index,
            isPoison = isPoison
        )
    )
}
```

- `forEachIndexed` entrega valor e índice.
- Los parámetros de la lambda son index e isPoison.
- Random.nextInt usa mínimo incluido y máximo excluido.
- toFloat convierte Int a Float.
- Cada índice resta otra separación vertical.

### Reaparición detrás de la cola

```
val objectBehindQueue = objects
    .asSequence()
    .filter { it != obj }
    .minOfOrNull { it.y }
    ?: INITIAL_SPAWN_Y
```

- `asSequence` procesa operaciones de forma perezosa.
- `it` es el nombre implícito del único parámetro de una lambda.
- `!=` compara identidad: excluye exactamente la misma instancia.
- `minOfOrNull` obtiene la Y más pequeña o null si no hay elementos.
- `?:` usa INITIAL\_SPAWN\_Y si el resultado fue null.

Luego se resta SPAWN\_SPACING\_WORLD. Por eso el objeto reaparece después del último y no encima de otro.

## 11. Colisiones explicadas

El juego usa AABB: Axis-Aligned Bounding Box. Son rectángulos sin rotación.

```
val touchesPlayer = viewportHeightDp > 0f &&
    objectLeft < playerLeft + PLAYER_HITBOX_WIDTH_DP &&
    objectLeft + OBJECT_HITBOX_WIDTH_DP > playerLeft &&
    objectTop + OBJECT_HITBOX_INSET_TOP_DP <
        playerTop + PLAYER_HITBOX_HEIGHT_DP &&
    objectTop + OBJECT_HITBOX_INSET_TOP_DP +
        OBJECT_HITBOX_HEIGHT_DP > playerTop
```

| Término            | Qué significa y cómo defenderlo                                           |
|--------------------|---------------------------------------------------------------------------|
| <b>Condición 1</b> | El lado izquierdo del objeto está antes del lado derecho del jugador.     |
| <b>Condición 2</b> | El lado derecho del objeto pasó el lado izquierdo del jugador.            |
| <b>Condición 3</b> | La parte superior del objeto está antes de la parte inferior del jugador. |
| <b>Condición 4</b> | La parte inferior del objeto pasó la parte superior del jugador.          |

Si las cuatro condiciones son verdaderas, existe superposición horizontal y vertical.

### Por qué hay insets

Los PNG ocupan una caja grande y contienen transparencia. Los insets mueven y reducen la hitbox hacia la parte visible. Esto evita recoger veneno cuando el dibujo parece pasar al lado.

```
if (touchesPlayer) {
    if (obj.isPoison) {
        poisonHits++
    } else {
        score++
    }
    resetObject(obj)
    return@forEach
}
```

- `++` incrementa en uno.
- `return@forEach` termina la iteración actual, no updateGame completa.
- Esto evita que el mismo objeto sea contado también como perdido en el mismo frame.

### Condiciones de derrota

- Tres venenos: playerState = dead e isGameOver = true.
- Cinco zanahorias perdidas: playerState = dead e isGameOver = true.

## 12. Canvas y rendimiento

```
Canvas(
    modifier = Modifier
        .fillMaxSize()
        .pointerInput(gameLogic) { ... }
) {
    gameLogic.frameVersion
    // drawImage(...)
}
```

- Canvas proporciona un DrawScope para dibujar directamente.
- fillMaxSize ocupa la pantalla disponible.
- pointerInput instala manejo de gestos.
- gameLogic es la clave; si cambia, se reinicia ese manejador.

### frameVersion

Los x/y internos de FallingObject son variables normales por rendimiento. Al final de cada update, frameVersion aumenta. Como se lee dentro del bloque de dibujo, Compose invalida el Canvas y solicita redraw sin reconstruir toda la pantalla.

```
drawImage(
    image = it,
    dstOffset = IntOffset(x, y),
    dstSize = IntSize(objectSize, objectSize)
)
```

| Término          | Qué significa y cómo defenderlo                           |
|------------------|-----------------------------------------------------------|
| <b>image</b>     | Bitmap que se dibuja.                                     |
| <b>dstOffset</b> | Coordenada entera de destino, esquina superior izquierda. |
| <b>dstSize</b>   | Ancho y alto en píxeles del rectángulo de destino.        |
| <b>density</b>   | Cantidad aproximada de píxeles físicos por dp.            |

**Por qué no diez Image:** Mover diez composables Image cada frame provoca más recomposición, medición y colocación. Un Canvas agrupa el trabajo en una sola fase de dibujo.

### Gestos

```
detectDragGestures { _, dragAmount ->
    onMovePlayer(dragAmount.x)
```

```
}
```

- El guion bajo ignora un parámetro que no se necesita.
- dragAmount es un desplazamiento desde el evento anterior.
- Solo se usa x porque el conejo se mueve horizontalmente.

## 13. Bitmaps y memoria

CatchFoodBitmaps es un data class privado que agrupa referencias a sprites ya decodificados.

```
private data class CatchFoodBitmaps(
    val player: ImageBitmap?,
    val food: ImageBitmap?,
    val poison: ImageBitmap?,
    ...
)
```

- Es private porque solo CatchFoodGame.kt necesita conocerlo.
- Cada propiedad es nullable para tolerar errores de assets.

### Decodificación en dos pasos

```
val bounds = BitmapFactory.Options().apply {
    inJustDecodeBounds = true
}
```

- apply ejecuta una lambda usando el objeto como receptor.
- inJustDecodeBounds lee dimensiones sin cargar todos los píxeles.

```
val options = BitmapFactory.Options().apply {
    inSampleSize = sampleSize
    inPreferredConfig = Bitmap.Config.ARGB_8888
}
```

- inSampleSize reduce ancho y alto durante decodificación.
- ARGB\_8888 usa 8 bits para alfa, rojo, verde y azul.
- Reducir resolución evita ocupar memoria con píxeles invisibles.

### runCatching

`runCatching { ... }.getOrNull()` convierte cualquier excepción del bloque en null. La UI puede omitir el sprite en vez de cerrarse.

## 14. Room y SQLite desde cero

**Respuesta esencial:** Room no es PostgreSQL. Room genera una capa segura sobre SQLite, una base local almacenada dentro del teléfono.

### Entity = tabla

```
@Entity(tableName = "catch_food_scores")
data class CatchFoodScoreEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0,
    val foodCaught: Int
)
```

| Término             | Qué significa y cómo defenderlo                    |
|---------------------|----------------------------------------------------|
| <b>@Entity</b>      | Anotación Room que convierte la clase en tabla.    |
| <b>tableName</b>    | Nombre real de la tabla SQLite.                    |
| <b>@PrimaryKey</b>  | Columna que identifica de forma única una fila.    |
| <b>autoGenerate</b> | SQLite asigna automáticamente el siguiente id.     |
| <b>Long</b>         | Entero de 64 bits, apropiado para identificadores. |

### DAO = operaciones

```
@Dao
interface CatchFoodScoreDao {
    @Insert
    suspend fun insert(score: CatchFoodScoreEntity)
}
```

- interface define un contrato sin implementación manual.
- Room/KSP genera la implementación.
- @Insert indica el SQL de inserción automáticamente.
- suspend permite suspender la corrutina durante I/O.

### Consulta SQL

```
SELECT foodCaught
FROM catch_food_scores
```

```
ORDER BY foodCaught DESC, id ASC  
LIMIT 10
```

| Término         | Qué significa y cómo defenderlo                      |
|-----------------|------------------------------------------------------|
| <b>SELECT</b>   | Columnas que se desean devolver.                     |
| <b>FROM</b>     | Tabla que se consulta.                               |
| <b>ORDER BY</b> | Orden de resultados.                                 |
| <b>DESC</b>     | De mayor a menor.                                    |
| <b>ASC</b>      | De menor a mayor; desempata por partida más antigua. |
| <b>LIMIT 10</b> | Devuelve como máximo diez filas.                     |

## 15. Database, Singleton y Repository

```
@Database(
    entities = [CatchFoodScoreEntity::class],
    version = 1,
    exportSchema = false
)
abstract class CatchFoodDatabase : RoomDatabase()
```

- `entities = [...]` registra las tablas.
- `::class` obtiene la referencia Kotlin a la clase.
- version identifica la versión del esquema.
- Si el esquema cambia, debe incrementarse y normalmente añadirse una migración.
- abstract permite que Room genere la implementación concreta.

### Singleton thread-safe

```
@Volatile
private var instance: CatchFoodDatabase? = null

fun getInstance(context: Context): CatchFoodDatabase =
    instance ?: synchronized(this) {
        instance ?: Room.databaseBuilder(...).build()
            .also { instance = it }
    }
```

| Término                   | Qué significa y cómo defenderlo                                               |
|---------------------------|-------------------------------------------------------------------------------|
| <b>@Volatile</b>          | Garantiza que los hilos vean el valor actualizado de instance.                |
| <b>synchronized</b>       | Solo un hilo puede ejecutar el bloque simultáneamente.                        |
| <b>?:</b>                 | Si instance existe la devuelve; si no, crea la base.                          |
| <b>also</b>               | Devuelve el mismo objeto y ejecuta una acción lateral: guardarlo en instance. |
| <b>applicationContext</b> | Contexto de larga vida; evita conservar una Activity.                         |

### Repository

```
class CatchFoodScoreRepository(context: Context) {
    private val scoreDao =
        CatchFoodDatabase.getInstance(context).scoreDao()

    suspend fun saveScore(score: Int) {
```



```
        scoreDao.insert(CatchFoodScoreEntity(foodCaught = score))
    }
}
```

El Repository ofrece operaciones con nombres del dominio y oculta DAO/Room al ViewModel. Esto permite cambiar la fuente de datos con menor impacto.

## 16. Histórico y estado asíncrono

```
val scores by produceState(
    initialValue = emptyList(),
    scoreRepository
) {
    value = scoreRepository.getTopScores()
}
```

- produceState devuelve State<List<Int>>.
- by permite usar scores directamente.
- emptyList es el valor mientras termina SQLite.
- scoreRepository es una clave del efecto.
- value actualiza el State y provoca recomposición.

### LazyColumn

```
LazyColumn {
    itemsIndexed(scores) { index, score ->
        Text("${index + 1}.")
        Text(score.toString())
    }
}
```

- LazyColumn es una lista vertical que compone elementos según se necesitan.
- itemsIndexed entrega posición y valor.
- Interpolación `\${...}` inserta expresiones en String.
- toString convierte Int a texto.

**Flujo completo:** Room devuelve List<Int> → produceState cambia scores → Compose recompone → LazyColumn dibuja ranking.

## 17. Audio y ciclo de vida

```
mediaPlayer = MediaPlayer().apply {
    context.assets.openFd("...mp3").use { fd ->
        setDataSource(fd.fileDescriptor, fd.startOffset, fd.length)
    }
    isLooping = true
    prepare()
    start()
}
```

- MediaPlayer reproduce audio nativo Android.
- openFd obtiene descriptor, desplazamiento y longitud del asset.
- setDataSource define el origen.
- isLooping repite la canción.
- prepare prepara sincrónicamente.
- start inicia reproducción.

```
mediaPlayer?.apply {
    if (isPlaying) stop()
    release()
}
mediaPlayer = null
```

- `?.` evita operar si mediaPlayer es null.
- release libera recursos nativos.
- Asignar null permite crear uno nuevo al volver.

**Defensa:** La limpieza está conectada a onDispose para evitar audio duplicado y fugas de recursos.

## 18. Recorrido completo de una partida

12. El NavHost muestra CatchFoodMenu como destino inicial.
13. CatchFoodScreen carga el fondo e inicia la música.
14. El usuario pulsa Jugar; la lambda llama navController.navigate.
15. CatchFoodGame obtiene CatchFoodViewModel mediante viewModel().
16. El ViewModel ya posee GameLogic y UiState inicial.
17. LaunchedEffect inicia el loop de frames.
18. withFrameNanos entrega el timestamp del nuevo frame.
19. Se calcula deltaSeconds y ViewModel llama updateGame.
20. GameLogic mueve objetos y aumenta gradualmente speed.
21. GameLogic calcula hitboxes y detecta colisiones.
22. El ViewModel sincroniza cambios visibles hacia UiState.
23. Compose actualiza contadores o la imagen del conejo.
24. frameVersion invalida el Canvas para nuevas posiciones.
25. El usuario arrastra; la lambda envía dragAmount.x al ViewModel.
26. Con tres venenos o cinco comidas perdidas se activa game over.
27. El ViewModel captura el puntaje y lanza una corrutina.
28. Repository crea Entity y DAO inserta en SQLite.
29. Al abrir Histórico, DAO ejecuta SELECT ORDER BY LIMIT 10.
30. produceState recibe la lista y LazyColumn muestra los resultados.

### La explicación de una sola zanahoria

Se crea con x aleatoria e y negativa. En cada frame y aumenta. Canvas convierte sus coordenadas y la dibuja. Cuando su hitbox se superpone con la del conejo, score aumenta, UiState se sincroniza, el Text se recompone y la zanahoria se coloca detrás de la cola.

## 19. Glosario rápido para preguntas sorpresa

### Conceptos en una frase

| Término               | Qué significa y cómo defenderlo                                                                 |
|-----------------------|-------------------------------------------------------------------------------------------------|
| <b>Anotación</b>      | Metadato con @ que informa a compiladores o librerías, por ejemplo @Composable, @Entity y @Dao. |
| <b>API</b>            | Contrato que permite que componentes se comuniquen; no significa necesariamente Internet.       |
| <b>Callback</b>       | Función entregada para ejecutarse cuando ocurra algo.                                           |
| <b>Lambda</b>         | Función anónima expresada entre llaves.                                                         |
| <b>Scope</b>          | Alcance donde existe un nombre o donde vive una corrutina.                                      |
| <b>Corrutina</b>      | Trabajo suspendible y liviano para operaciones concurrentes.                                    |
| <b>Suspend</b>        | Función que puede pausar su ejecución sin bloquear el hilo.                                     |
| <b>Hilo principal</b> | Hilo que procesa UI; bloquearlo provoca congelamiento o ANR.                                    |
| <b>Estado</b>         | Información mutable que determina comportamiento o apariencia.                                  |
| <b>UiState</b>        | Fotografía inmutable de los datos visibles de una pantalla.                                     |
| <b>Recomposición</b>  | Reejecución selectiva de Composables por cambios observados.                                    |
| <b>Inmutabilidad</b>  | Crear una nueva instancia en vez de alterar la anterior.                                        |
| <b>Acoplamiento</b>   | Grado de dependencia entre componentes.                                                         |
| <b>Encapsulación</b>  | Ocultar detalles y exponer solo operaciones necesarias.                                         |
| <b>Singleton</b>      | Patrón que mantiene una única instancia compartida.                                             |
| <b>Repository</b>     | Abstracción que oculta las fuentes y operaciones de datos.                                      |
| <b>DAO</b>            | Interfaz de acceso a datos con operaciones de lectura/escritura.                                |

| <b>Término</b>      | <b>Qué significa y cómo defenderlo</b>                        |
|---------------------|---------------------------------------------------------------|
| <b>Entity</b>       | Clase que Room transforma en tabla.                           |
| <b>Persistencia</b> | Datos que sobreviven al cierre de la aplicación.              |
| <b>SQLite</b>       | Base relacional embebida dentro del dispositivo.              |
| <b>KSP</b>          | Procesador de símbolos que genera código de Room al compilar. |
| <b>Canvas</b>       | Superficie de dibujo directo para gráficos eficientes.        |
| <b>FPS</b>          | Frames dibujados por segundo.                                 |
| <b>Delta time</b>   | Tiempo transcurrido desde la actualización anterior.          |
| <b>Hitbox</b>       | Área matemática usada para colisiones.                        |
| <b>AABB</b>         | Colisión entre rectángulos alineados con los ejes.            |
| <b>Nullable</b>     | Tipo que admite null.                                         |
| <b>Leak</b>         | Recurso retenido cuando ya debería liberarse.                 |

## 20. Banco de preguntas de defensa

**P: ¿Qué patrón de arquitectura utilizaste?**

**R: Una separación inspirada en MVVM con Repository.**

*Si piden más detalle: Compose actúa como View, CatchFoodViewModel administra UiState y eventos, GameLogic contiene reglas y Repository abstrae Room.*

**P: ¿Qué es una lambda?**

**R: Una función anónima que se puede pasar como dato.**

*Si piden más detalle: Los botones reciben lambdas onClick y las pantallas reciben callbacks de navegación.*

**P: ¿Qué es UiState?**

**R: Un objeto inmutable con todo lo que la pantalla necesita mostrar.**

*Si piden más detalle: CatchFoodUiState contiene puntajes, estado del jugador, pausa y derrota.*

**P: ¿Por qué ViewModel?**

**R: Para separar lógica/estado de la UI y conservarlos ante recomposiciones.**

*Si piden más detalle: También ofrece viewModelScope para guardar en Room sin bloquear la interfaz.*

**P: ¿Qué es recomposición?**

**R: La reejecución selectiva de UI cuando cambia un State observado.**

**P: ¿Por qué Canvas?**

**R: Para redibujar objetos animados eficientemente en una sola superficie.**

*Si piden más detalle: Evita medir y colocar diez Image composables en cada frame.*

**P: ¿Por qué delta time?**

**R: Para que la velocidad dependa del tiempo real y no de los FPS.**

**P: ¿Cómo detectás colisión?**

**R: Con cuatro comparaciones AABB entre hitboxes rectangulares reducidas.**

**P: ¿Por qué la hitbox es menor que el PNG?**

**R: Porque el PNG contiene transparencia y una caja visual mayor al personaje real.**

**P: ¿Cómo evitás objetos amontonados?**

**R: Al reaparecer, cada objeto se coloca detrás de la menor coordenada Y con separación fija.**

**P: ¿Qué es Room?**

**R: Una capa oficial sobre SQLite local con validación y generación de código.**

**P: ¿Usa PostgreSQL?**

**R: No. CatchFood usa SQLite dentro del teléfono.**

**P: ¿Qué hace el DAO?**

**R:** Define inserciones y consultas SQL de la tabla de puntajes.

**P: ¿Qué hace Repository?**

**R:** Oculta Room al ViewModel y expone operaciones del dominio.

**P: ¿Por qué suspend?**

**R:** Porque el acceso a almacenamiento puede esperar y no debe bloquear la UI.

**P: ¿Qué hace KSP?**

**R:** Genera en compilación la implementación necesaria de Room.

**P: ¿Por qué Singleton para Database?**

**R:** Para reutilizar una única conexión segura en el proceso.

**P: ¿Qué hace @Volatile?**

**R:** Hace visible entre hilos el valor más reciente de la instancia.

**P: ¿Qué hace synchronized?**

**R:** Evita que dos hilos creen la base simultáneamente.

**P: ¿Por qué scoreSavedForCurrentGame?**

**R:** Evita insertar la misma partida más de una vez.

**P: ¿Qué diferencia hay entre remember y ViewModel?**

**R:** remember conserva durante recomposición; ViewModel está diseñado para estado de pantalla y cambios de configuración.

**P: ¿Qué hace DisposableEffect?**

**R:** Conecta creación y limpieza de un recurso al ciclo de composición.

**P: ¿Qué mejora harías?**

**R:** Usaría enum para playerState y Flow para observar el ranking automáticamente.

*Si piden más detalle: También añadiría migraciones de Room y pruebas unitarias para colisiones/spawn.*



## 21. Simulacro oral

### Presentación de 3 minutos

Mi módulo se llama CatchFood. Está construido con Jetpack Compose y tiene tres pantallas internas: menú, juego e histórico. La navegación está centralizada en CatchFoodNavHost y las pantallas reciben callbacks para no depender directamente del NavController.

Durante el juego, CatchFoodViewModel coordina el estado visible y una clase CatchFoodGameLogic que contiene las reglas. CatchFoodUiState es una data class inmutable con score, golpes de veneno, comida perdida, estado visual, pausa y game over. La UI observa este objeto y se recompone únicamente cuando cambian valores visibles.

El loop usa withFrameNanos para sincronizarse con Android. Calculo delta time para obtener movimiento consistente entre dispositivos. Los objetos se dibujan en un Canvas para reducir recomposición y layout. Las colisiones son AABB con hitboxes reducidas para ajustarse al contenido visible de los PNG.

Cuando termina la partida, el ViewModel captura el puntaje una sola vez y lo guarda con una corrutina. El Repository utiliza Room sobre SQLite. La Entity representa la tabla, el DAO define inserción y consulta, y el histórico ejecuta un ORDER BY descendente con LIMIT 10. Finalmente, produceState convierte la consulta suspendida en estado observable y LazyColumn muestra el ranking.

### Si te interrumpen

- Si preguntan UI: habla de Compose, State y recomposición.
- Si preguntan rendimiento: habla de Canvas, frameVersion, sampling y delta time.
- Si preguntan arquitectura: habla de callbacks, ViewModel, UiState y Repository.
- Si preguntan base: Entity, DAO, Database, Repository y SQLite.
- Si preguntan seguridad de recursos: remember, DisposableEffect y release.

### Errores que no debes decir

- No digás que Room es PostgreSQL.
- No digás que una corrutina es exactamente un hilo.
- No digás que remember guarda datos después de cerrar la app.
- No digás que Canvas evita toda recomposición; reduce el trabajo animado y separa redraw de layout.
- No digás que ViewModel dibuja la UI; administra estado y acciones.

## 22. Hoja de repaso final

**Lambda:** función anónima pasada como dato: `{ onGoToGame() }`.

**Callback:** lambda que otro componente llama cuando ocurre un evento.

**UiState:** fotografía inmutable de valores visibles.

**ViewModel:** coordina estado, eventos, motor y persistencia.

**Composable:** función declarativa que describe interfaz.

**Recomposición:** actualización selectiva por cambios en State.

**remember:** conserva un cálculo/objeto entre recomposiciones.

**Corrutina:** trabajo suspendible que evita bloquear UI.

**Room:** capa sobre SQLite local.

**DAO:** contrato de acceso y consultas a datos.

**Canvas:** dibujo directo eficiente para la escena animada.

**Delta time:** tiempo real entre frames.

**AABB:** superposición de rectángulos para colisiones.

### Diez frases que tenés que memorizar

31. La UI envía eventos y observa estado; no ejecuta SQL.
32. El ViewModel coordina el juego y sobrevive a recomposiciones.

33. UiState agrupa datos visibles en una data class inmutable.
34. GameLogic contiene reglas, posiciones, colisiones y spawn.
35. withFrameNanos sincroniza el loop con Android.
36. Delta time mantiene velocidad consistente entre dispositivos.
37. Canvas reduce el trabajo de animar múltiples elementos Compose.
38. Las hitboxes reducidas evitan capturas visualmente falsas.
39. Room usa SQLite local y KSP genera implementaciones.
40. El Repository desacopla ViewModel de DAO y Database.

**Cierre recomendado:** El diseño separa responsabilidades, optimiza el renderizado en Android real y conserva el historial local sin bloquear el hilo principal.